

Mobile Shop Management System

Current Project Report & Planning Baseline

Domain: Web Software / Retail Management

Academic Context: MCA ADBMS(S) project

Business Model: Software designed for one individual mobile shop

Purpose

A small web-based application for managing mobile inventory, stock, customers, suppliers, sales and billing, while demonstrating selected data structures, algorithms and database concepts. The project is intentionally kept minimal to make implementation, testing and viva presentation manageable.

1. Technology Stack

Layer	Technology
Frontend	HTML + CSS
Client-side scripting	JavaScript, only where useful
Backend	Python Flask
Database	MySQL
Database connectivity	MySQL Connector/Python
Algorithms	Python
Development environment	VS Code
Local server	Flask development server

2. System Architecture

The proposed architecture is a simple three-layer web application: HTML/CSS/JavaScript for the interface, Flask/Python for application logic and algorithms, and MySQL for persistent data.

USER → HTML/CSS/JS → FLASK/PYTHON → MYSQL

Within Flask, business logic, authentication, CRUD operations, BFS, Merge Sort and Greedy processing are handled before communicating with MySQL.

3. Database Scope

The project deliberately uses exactly four core business tables. No separate Admin, Bill, Stock, Brand, Category, Payment, Cart or Shop table is currently planned.

CUSTOMER • MOBILE • SALES • SUPPLIER

4. Database Tables

Customer

Attribute	Type	Key / Constraint
customer_id	INT	PK, AUTO_INCREMENT
customer_name	VARCHAR(100)	NOT NULL

Attribute	Type	Key / Constraint
address	VARCHAR(255)	NOT NULL
phone	VARCHAR(15)	NOT NULL, UNIQUE
username	VARCHAR(50)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL; store hash

Supplier

Attribute	Type	Key / Constraint
supplier_id	INT	PK, AUTO_INCREMENT
supplier_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(15)	NOT NULL, UNIQUE
username	VARCHAR(50)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL; store hash

Mobile

Attribute	Type	Key / Constraint
mobile_id	INT	PK, AUTO_INCREMENT
brand	VARCHAR(50)	NOT NULL
model	VARCHAR(100)	NOT NULL
price	DECIMAL(10,2)	NOT NULL
stock	INT	NOT NULL, DEFAULT 0
supplier_id	INT	FK → supplier.supplier_id, NOT NULL

Sales

Attribute	Type	Key / Constraint
sales_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK → customer.customer_id, NOT NULL
mobile_id	INT	FK → mobile.mobile_id, NOT NULL
quantity	INT	NOT NULL
amount	DECIMAL(10,2)	NOT NULL
sales_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
warranty	VARCHAR(50)	NOT NULL

5. Relationships

Relationship	Cardinality	Meaning
SUPPLIER → MOBILE	1 : N	One supplier can supply many mobiles
CUSTOMER → SALES	1 : N	One customer can have many sales
MOBILE → SALES	1 : N	One mobile model can appear in many sales

Conceptually, Customer and Mobile have an M:N purchase relationship. In the relational implementation this is resolved through the Sales entity: CUSTOMER 1:N SALES N:1 MOBILE. Sales stores transaction-specific attributes such as quantity, amount, date and warranty.

6. Chen ER Diagram Plan

Entities: CUSTOMER, MOBILE, SALES, SUPPLIER.

Relationships: SUPPLIES, MAKES, SOLD IN.

Chen symbols: rectangle = entity; oval = attribute; underlined oval = primary key; diamond = relationship; 1/N = cardinality.

Supplier 1 — SUPPLIES — N Mobile

Customer 1 — MAKES — N Sales

Mobile 1 — SOLD IN — N Sales

The foreign-key attributes supplier_id, customer_id and mobile_id are represented in the relational schema. In pure Chen notation, the relationships communicate the associations.

7. Authentication Concept

The application represents one individual shop, so an Admin database table is not required. Admin is treated as an application role. Admin credentials can be stored in Flask configuration or environment variables. Customer and Supplier credentials are stored in their respective tables.

Authentication answers 'Who is the user?'. Authorization answers 'What is the user allowed to do?'. Flask sessions can maintain login state and role-based route protection can prevent unauthorized access.

8. Modules

Module	Main functions
Admin	Dashboard; add/view/update/delete mobiles; update stock; view sales
Customer	Login; own profile; browse mobiles; search/filter/sort; buy; purchase history; bill
Supplier	Login; own profile; add mobile; view supplied mobiles; update mobile/stock

9. Search, Filter and Sort

Search: Mobile ID, brand and model.

Filter: brand, price range and availability.

Sort: price ascending/descending, stock ascending/descending, brand/model A–Z or Z–A.

10. Data Structures and Algorithms

Requirement	Implementation	Purpose / Justification	Complexity
Graph + BFS	Related Mobile Explorer	Explore related mobiles level-by-level	$O(V + E)$
Merge Sort	Mobile sorting	Demonstrate efficient sorting of mobile records	$O(n \log n)$
Greedy	Best Mobile Within Budget	Select highest-priced available mobile not exceeding budget	Project-specific greedy strategy

The algorithms are intended to be integrated into meaningful application features rather than added as isolated demonstration buttons. The Greedy feature will be described as a project-specific budget-selection strategy rather than claiming greedy is universally optimal.

11. Purchase and Billing Workflow

Customer login → browse mobiles → search/filter/sort → select mobile → enter quantity → check stock → calculate amount → confirm purchase → insert Sales record → reduce Mobile stock → generate bill.

Amount = mobile price × quantity.

No separate Bill table is required. A bill can be generated dynamically using Customer + Sales + Mobile data.

12. Transaction Handling

A purchase changes both Sales and Mobile stock. These operations should be handled as a single database transaction:

```
START TRANSACTION → check stock → INSERT sales → UPDATE mobile stock → COMMIT
```

If an operation fails, ROLLBACK should be used to prevent inconsistent data such as a recorded sale without a corresponding stock reduction.

13. ADBMS Concepts to Demonstrate

DDL: CREATE, ALTER. DML: INSERT, UPDATE, DELETE. DQL: SELECT. Primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, JOINS, WHERE, ORDER BY, GROUP BY, COUNT/SUM/AVG and transactions can all be demonstrated using the existing four-table design.

14. Proposed Flask Project Structure

```
mobile_shop/ ■■■ app.py ■■■ database.py ■■■ algorithms/ ■ ■■■ bfs.py ■ ■■■ merge_sort.py ■ ■■■
greedy.py ■■■ templates/ ■ ■■■ base.html ■ ■■■ login.html ■ ■■■ admin/ ■ ■■■ customer/ ■ ■■■
supplier/ ■■■ static/ ■■■ css/ ■■■ js/
```

15. Features Explicitly Excluded

Online payment, delivery tracking, reviews, ratings, offers/coupons, wishlist, SMS/email notifications, AI recommendations, multi-shop support, separate inventory/bill/admin/shop tables, product categories, complex analytics, microservices, REST API requirements, React/Angular and Java/Spring are outside the current scope.

16. Current Status

Finalized	Still to finalize
Project title and individual-shop concept	Exact registration/login workflow
Flask + MySQL + HTML/CSS	Supplier permissions
Four core tables	Mobile uniqueness rule
Admin as application role; no Admin table	Warranty representation
Customer/Supplier profile concept	Who is allowed to update stock
Admin, Customer and Supplier modules	Final ER diagram layout
Search/filter/sort	
BFS + Merge Sort + Greedy	
Sales, stock, billing, transactions	

17. Recommended Development Sequence

1. Finalize functional requirements and authentication rules.
2. Finalize database attributes, constraints and SQL schema.
3. Complete the Chen ER diagram.
4. Prepare Use-Case and DFD Level 0/Level 1 diagrams.
5. Design BFS, Merge Sort and Greedy components.
6. Set up Flask and MySQL connectivity.
7. Implement authentication and role-based access.

8. Implement CRUD and inventory management.
9. Implement search, filtering and sorting.
10. Implement sales, stock transaction and billing.
11. Test database operations, algorithms, authorization and input validation.
12. Prepare final documentation and presentation material.

Project baseline: A deliberately small Flask + MySQL web application for a single mobile shop, using four relational tables, three user modules, basic authentication/authorization, inventory and sales management, billing, search/filter/sort, and integrated BFS, Merge Sort and Greedy algorithms.